AI Travel Architect: Project Deep Dive

A Stateful Multi-Stage Agentic Workflow

Martina Speciale

Travel-Agent-AI

February 16, 2026

Project Goal and Scope

Objective

Build an autonomous travel planner that turns a user request (destination, dates, budget, interests, companion) into a realistic itinerary with verifiable places and actionable outputs.

Core requirements

- Persistent state across reasoning steps (not a single prompt call).
- Grounding on real-world data (places and flight options).
- Self-correction loop when the plan is inconsistent.
- Human-in-the-Loop (HITL) when confidence is low.
- Deliverables in multiple formats: terminal, HTML, and DOCX.

Architecture at a Glance

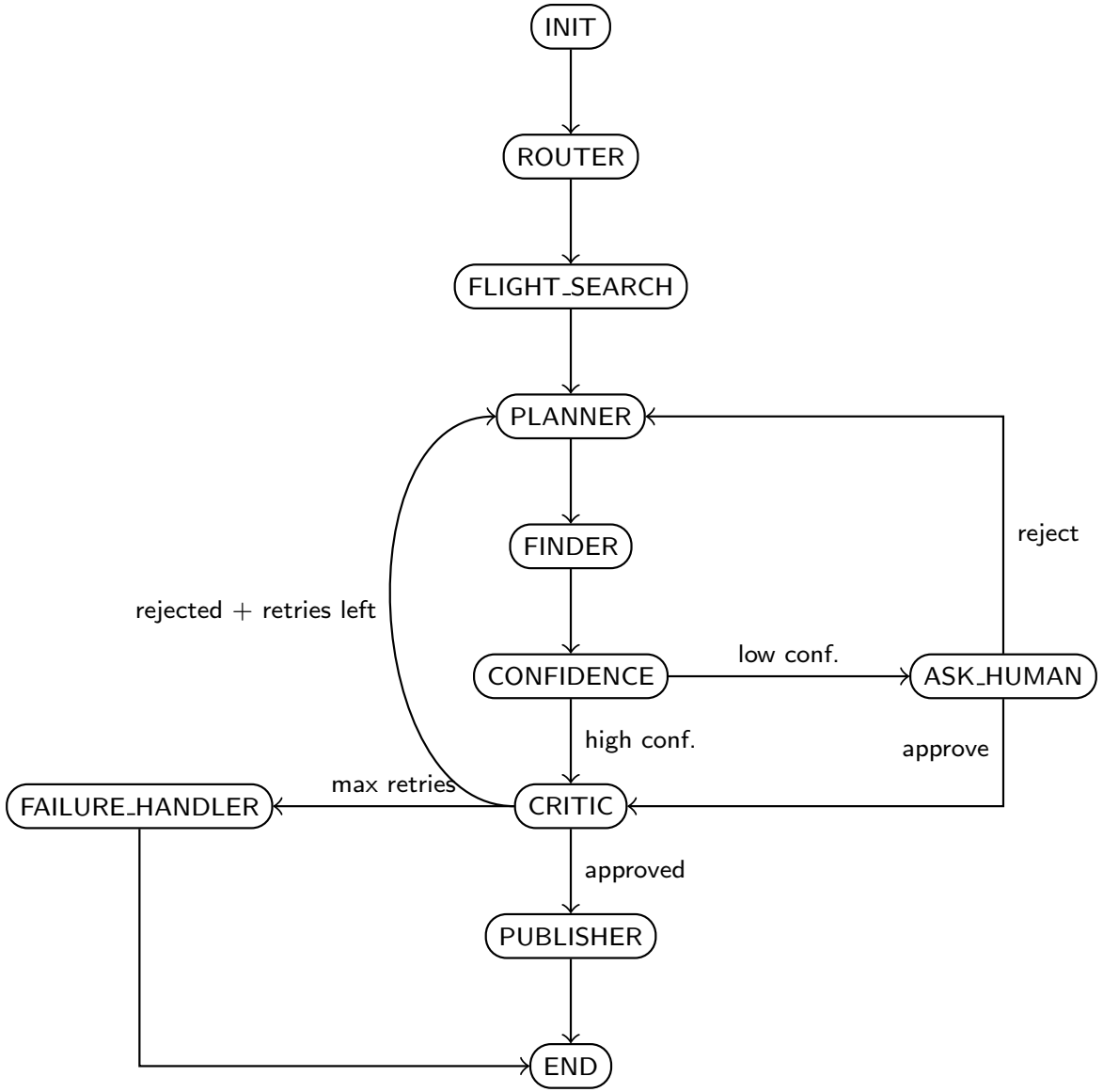
Implementation stack

- Orchestration: **LangGraph**
- LLM policy: **Groq** (+ optional fallback)
- Place grounding: **Google Maps Places API**
- Flight grounding: **SerpApi (Google Flights)**
- Route normalization: **local city→IATA seed CSV**
- Artifacts: **HTML + DOCX publisher**

Code map

- State schema: `app/core/state.py`
- Node logic: `app/engine/nodes.py`
- Graph wiring: `app/graph.py`
- Tools: `app/tools/*.py`
- Observability: `app/core/logger.py`

Execution Graph (LangGraph Flow)



Flow Commentary: What Each Node Does

1. **INIT** — collects user constraints (destination, dates, budget, companion) and initializes shared state.
2. **ROUTER** — infers travel style and normalizes intent for downstream planning.
3. **FLIGHT_SEARCH** — resolves city→IATA, proposes best outbound/return flight, asks user confirmation.
4. **PLANNER** — generates a day-by-day draft itinerary coherent with style, dates, and budget signal.
5. **FINDER** — grounds each proposed place via Google Maps (address/rating), marking verified vs non-verified.
6. **CONFIDENCE** — computes reliability score from verified-place ratio; if low (< 0.7), triggers HITL.
7. **ASK_HUMAN** — interruption node: user can approve continuation or reject and force re-planning.
8. **CRITIC** — validates feasibility and consistency (logistics + budget plausibility); approves or sends feedback.
9. **PUBLISHER** — generates final artifacts (terminal, HTML, DOCX) once approved.
10. **FAILURE_HANDLER** — safe exit after max retries, with explicit failure reason.

Shared Memory: TravelAgentState

Persistent graph state (TypedDict)

- Input: `user_input`, `destination`, `interests`, `budget`, `budget_total`, `companion`, `origin`, `depart_date`, `return_date`
- `days`: auto-computed from dates when possible, otherwise requested explicitly
- Planning output: `travel_style`, `itinerary`
- Flight output: `flight_options`, `flight_summary`, `flight_confidence_score`
- Control fields: `confidence_score`, `critic_feedback`, `retry_count`, `is_approved`
- Context fields: `banned_places`, `budget_context`

Why this matters

The state is the memory backbone of the system: each node reads/writes a shared object, enabling trajectory-level reasoning and deterministic routing decisions.

Node Responsibilities

Decision nodes

- **Router**: validates and normalizes user request.
- **Flight_Search**: resolves route and proposes outbound/return option.
- **Planner**: generates day-by-day itinerary draft.
- **Finder**: enriches draft with real places and ratings.
- **Critic**: checks feasibility (timing, distances, budget).

Control nodes

- **Confidence**: estimates reliability after grounding.
- **Ask_Human**: interruption point for approval/edits.
- **Publisher**: renders final itinerary to artifacts.
- **Failure_Handler**: safe termination after retries.

Grounding and Tooling Strategy

Real-world grounding pipeline

Plan text is converted into concrete places, then validated with external data.

- **Google Maps Tool**: resolves locations, addresses, ratings.
- **SerpApi Flights Tool**: retrieves structured flight options (one-way/return).
- **IATA seed resolver**: maps city names (IT/EN variants) to airport codes.
- **Robust tool interface**: tools return `list | error_string`; nodes apply safe fallback on errors (no crashes).
- **Confidence signal**: based on verified vs non-verified places ratio.

Failure mode addressed

Without grounding, LLMs can output plausible but non-existent places. Tool-verified retrieval reduces hallucinations and improves itinerary trust.

Self-Correction and HITL

Quality loop

1. Planner proposes itinerary.
2. Critic checks consistency and constraints.
3. If rejected, planner retries with critic feedback.
4. After max retries, graph exits via failure handler.

Human-in-the-Loop trigger

- If `confidence_score < 0.7`, execution pauses for human confirmation.
- User can approve (continue) or reject (re-plan).

Observability and Outputs

Observability

- Structured, color-coded logs for node actions.
- Trace of *thought vs action* transitions.
- Easier debugging of trajectory failures.

Delivered artifacts

- Terminal itinerary summary.
- **HTML report** with map-oriented layout.
- **DOCX report** for editable handoff.

Practical payoff

The output is not only text generation: it is a reproducible, inspectable, and user-ready travel artifact.

Limitations and Next Iteration

Current limitations

- API dependency (availability, quotas, response drift).
- Confidence score is heuristic, not fully calibrated.
- Critic quality depends on prompt quality and itinerary structure.

Planned improvements

- Automatic evaluation set (golden prompts + regression checks).
- Better international city/airport coverage in the local seed.
- Stronger date parsing and calendar constraints.
- Optional cost signals from richer place metadata.
- Native timeline map in final HTML output.

Lessons Learned

Key implementation lessons

- **Deterministic gates beat self-reported confidence:** moving confidence to verified-place ratio made routing more stable.
- **Fail-soft tooling is mandatory:** tool errors must degrade gracefully (fallback + continue), not crash the graph.
- **Input normalization is essential:** city→IATA mapping and date normalization need strict guards to avoid silent misrouting.
- **Retry loops need hard boundaries:** explicit retry caps and failure exits prevent planner-critic dead loops.

Practical takeaway

Reliability came from control logic and state discipline more than from prompt wording alone.

Takeaways

What this project demonstrates

A production-oriented agent is a system, not a single model call: **policy + controller + state + tools + evaluation loop**.

- Stateful orchestration enables robust multi-step reasoning.
- External tools provide grounding and reduce hallucinations.
- Critic/HITL loops improve reliability under uncertainty.
- Observability turns debugging into an engineering workflow.